

Grundlagen der Programmierung:

9. Objektorientierung II

Tutoren:

Daniel Grillmeyer, Ludwig Friedl, Jan Kerber, Jennifer Muck

1. Objektorientierung - Teil II
 1. Vererbung
 2. Polymorphismus
 3. Vorteile der Vererbung
2. Lösung: Schriftliches Aufgabenblatt 2

Rückblick: Die Tiere

- Wir wollen folgende Tiere simulieren:



Eisbaer



Katze



Dromedar



Hund



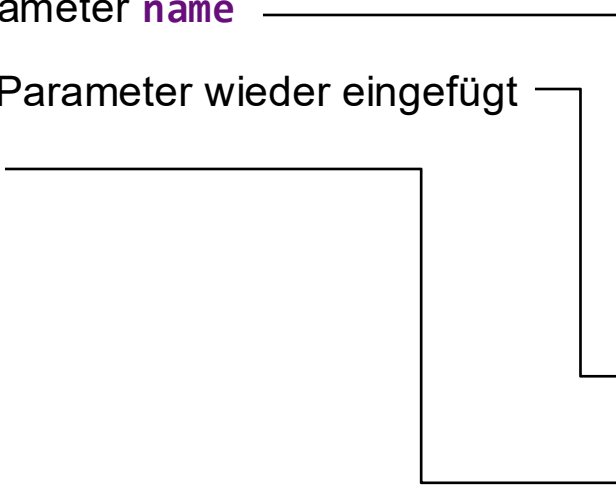
Pinguin

Rückblick: Die Tiere

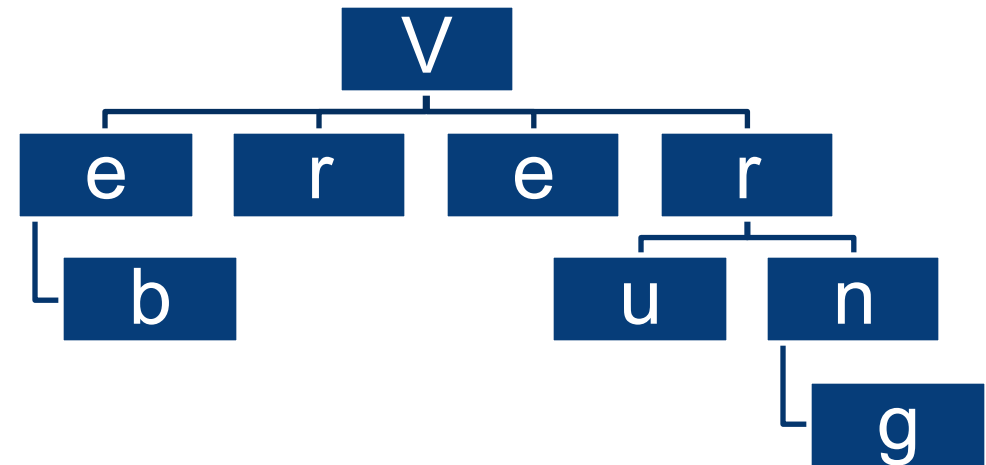
➤ Unser Stand von der letzten Übung:

- Attribut **name** vom Typ String für jede Tierklasse (Hund, Katze, Pinguin, ...) implementiert
- In der Klasse Hund implementiert:
 - Konstruktor mit String Parameter **name**
 - Default-Konstruktor ohne Parameter wieder eingefügt
 - Instanzmethode **friss()**

```
1 public class Hund {  
2  
3     String name;  
4  
5     Hund(String name) {  
6         this.name = name;  
7     }  
8  
9     Hund() { }  
10  
11     public void friss() {  
12         System.out.println(name + ": *mampf*");  
13     }  
14 }
```

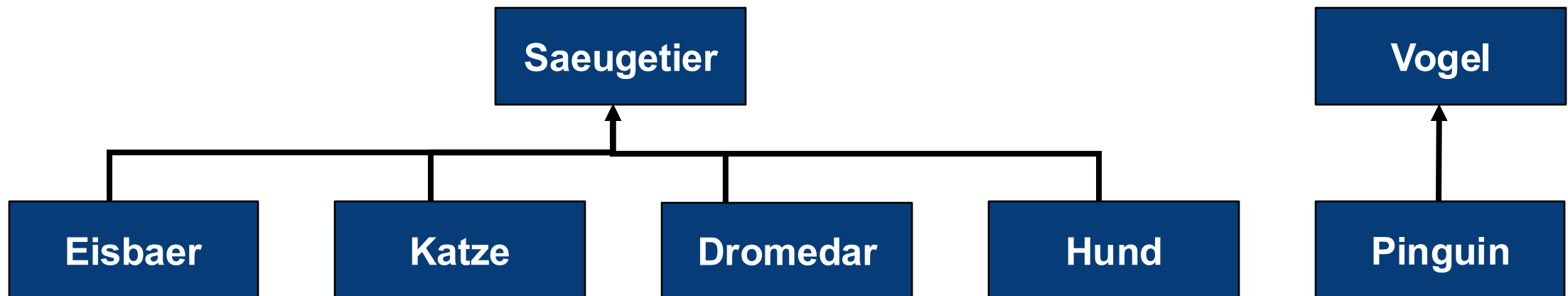


VERERBUNG



Säugetiere und Vögel

- Unsere Tiere lassen sich in zwei Gruppen unterteilen: Säugetiere und Vögel.
- Wir realisieren diese Gruppierung durch Vererbung



- Erstellen Sie die neuen Klassen und implementieren Sie die dargestellten Vererbungen

Säugetiere und Vögel

Vererbung am Beispiel der Klasse Eisbaer:

```
1 public class Eisbaer extends Saeugetier {  
2     ...  
3 }
```

Die Vererbung erlaubt es uns, die Kind-Klasse an Stelle der Eltern-Klasse zu verwenden:

```
1 Saeugetier s = new Eisbaer();
```

Puzzler: Vererbung

➤ Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();  
2 Katze katze1 = new Katze();  
3 Saeugetier saeugetier1 = new Saeugetier();  
4 Saeugetier saeugetier2 = new Hund();  
5 Hund hund2 = new Saeugetier();  
6 Hund hund3 = new Katze();  
7 Saeugetier saeugetier3 = new Katze();  
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

6

7

8

Puzzler: Vererbung

➤ Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();  
2 Katze katze1 = new Katze();  
3 Saeugetier saeugetier1 = new Saeugetier();  
4 Saeugetier saeugetier2 = new Hund();  
5 Hund hund2 = new Saeugetier();  
6 Hund hund3 = new Katze();  
7 Saeugetier saeugetier3 = new Katze();  
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

6

7

8

Puzzler: Vererbung

➤ Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();  
2 Katze katze1 = new Katze();  
3 Saeugetier saeugetier1 = new Saeugetier();  
4 Saeugetier saeugetier2 = new Hund();  
5 Hund hund2 = new Saeugetier();  
6 Hund hund3 = new Katze();  
7 Saeugetier saeugetier3 = new Katze();  
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

6

7

8

Puzzler: Vererbung

- Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();  
2 Katze katze1 = new Katze();  
3 Saeugetier saeugetier1 = new Saeugetier();  
4 Saeugetier saeugetier2 = new Hund();  
5 Hund hund2 = new Saeugetier();  
6 Hund hund3 = new Katze();  
7 Saeugetier saeugetier3 = new Katze();  
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

6

7

8

Puzzler: Vererbung

➤ Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();  
2 Katze katze1 = new Katze();  
3 Saeugetier saeugetier1 = new Saeugetier();  
4 Saeugetier saeugetier2 = new Hund();  
5 Hund hund2 = new Saeugetier();  
6 Hund hund3 = new Katze();  
7 Saeugetier saeugetier3 = new Katze();  
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

6

7

8

Puzzler: Vererbung

- Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();
2 Katze katze1 = new Katze();
3 Saeugetier saeugetier1 = new Saeugetier();
4 Saeugetier saeugetier2 = new Hund();
5 Hund hund2 = new Saeugetier();
6 Hund hund3 = new Katze();
7 Saeugetier saeugetier3 = new Katze();
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

6

7

8

Puzzler: Vererbung

- Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();  
2 Katze katze1 = new Katze();  
3 Saeugetier saeugetier1 = new Saeugetier();  
4 Saeugetier saeugetier2 = new Hund();  
5 Hund hund2 = new Saeugetier();  
6 Hund hund3 = new Katze();  
7 Saeugetier saeugetier3 = new Katze();  
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

6

7

8

Puzzler: Vererbung

- Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();  
2 Katze katze1 = new Katze();  
3 Saeugetier saeugetier1 = new Saeugetier();  
4 Saeugetier saeugetier2 = new Hund();  
5 Hund hund2 = new Saeugetier();  
6 Hund hund3 = new Katze();  
7 Saeugetier saeugetier3 = new Katze();  
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

6

7

8

Puzzler: Vererbung

- Welche Zuweisungen sind erlaubt?

```
1 Hund hund1 = new Hund();  
2 Katze katze1 = new Katze();  
3 Saeugetier saeugetier1 = new Saeugetier();  
4 Saeugetier saeugetier2 = new Hund();  
5 Hund hund2 = new Saeugetier();  
6 Hund hund3 = new Katze();  
7 Saeugetier saeugetier3 = new Katze();  
8 Saeugetier saeugetier4 = new Pinguin();
```

1

2

3

4

5

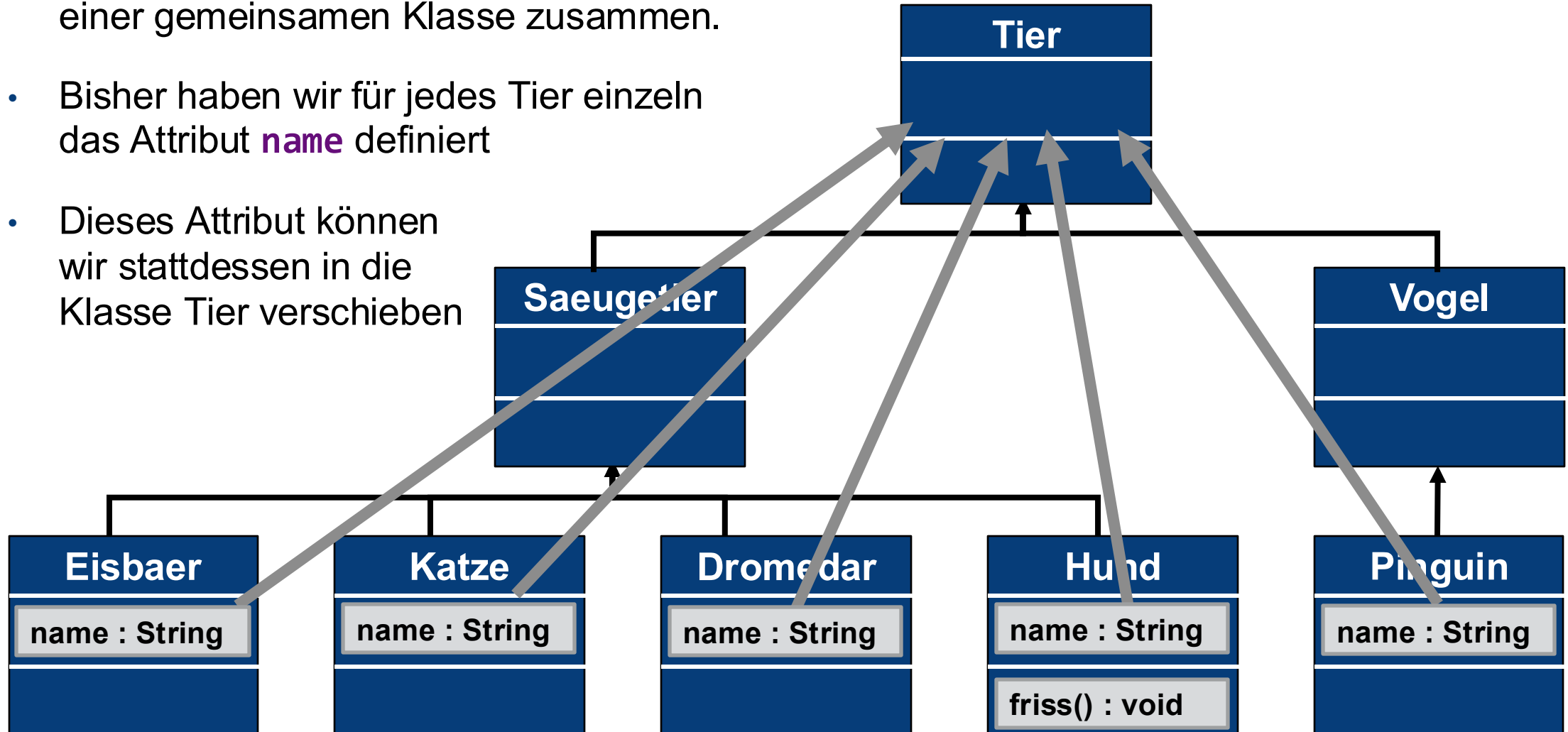
6

7

8

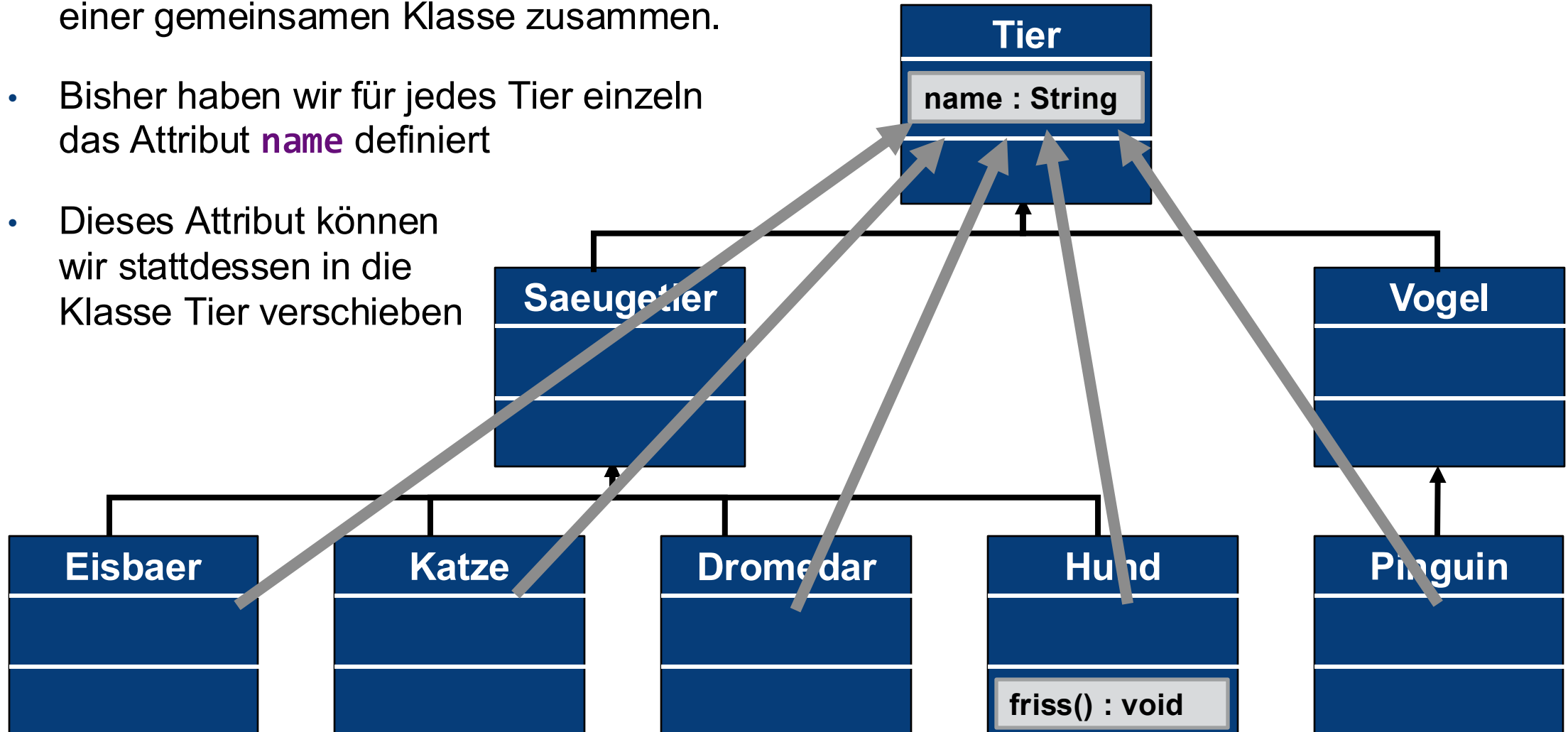
Noch eine Ebene

- Da alle Tiere gemeinsame Eigenschaften und Fähigkeiten besitzen fassen wir diese zu einer gemeinsamen Klasse zusammen.
- Bisher haben wir für jedes Tier einzeln das Attribut **name** definiert
- Dieses Attribut können wir stattdessen in die Klasse Tier verschieben



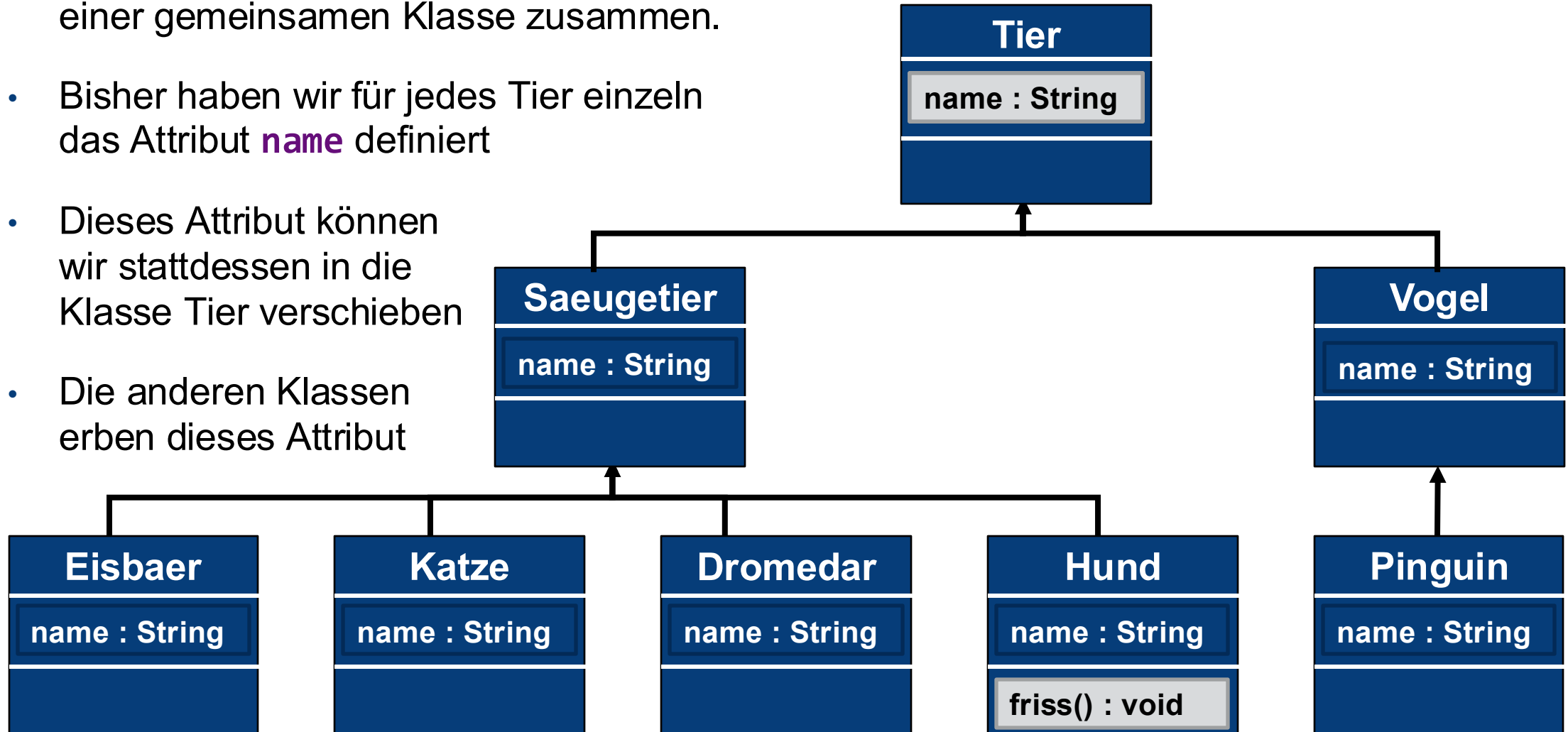
Noch eine Ebene

- Da alle Tiere gemeinsame Eigenschaften und Fähigkeiten besitzen fassen wir diese zu einer gemeinsamen Klasse zusammen.
- Bisher haben wir für jedes Tier einzeln das Attribut **name** definiert
- Dieses Attribut können wir stattdessen in die Klasse Tier verschieben



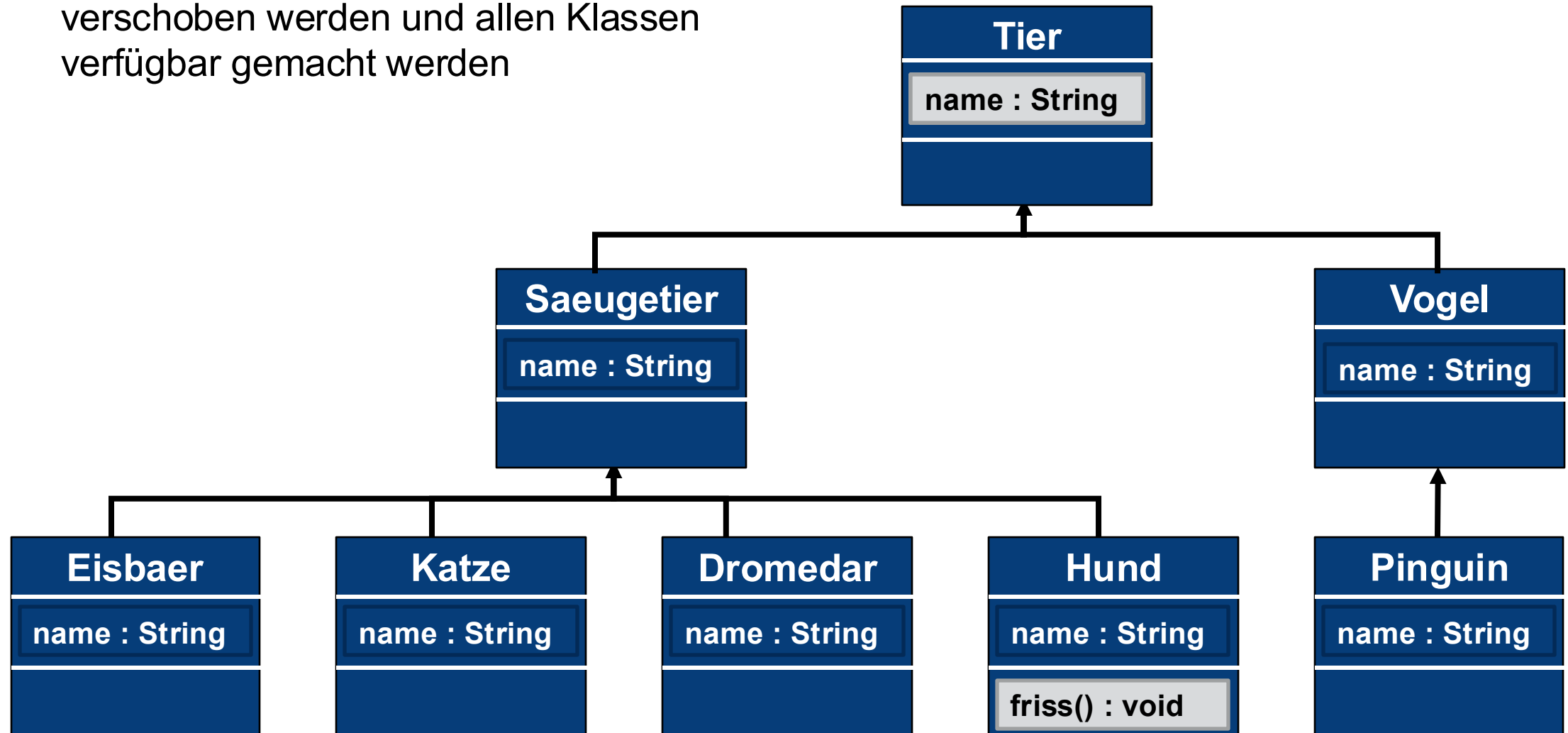
Noch eine Ebene

- Da alle Tiere gemeinsame Eigenschaften und Fähigkeiten besitzen fassen wir diese zu einer gemeinsamen Klasse zusammen.
- Bisher haben wir für jedes Tier einzeln das Attribut **name** definiert
- Dieses Attribut können wir stattdessen in die Klasse Tier verschieben
- Die anderen Klassen erben dieses Attribut



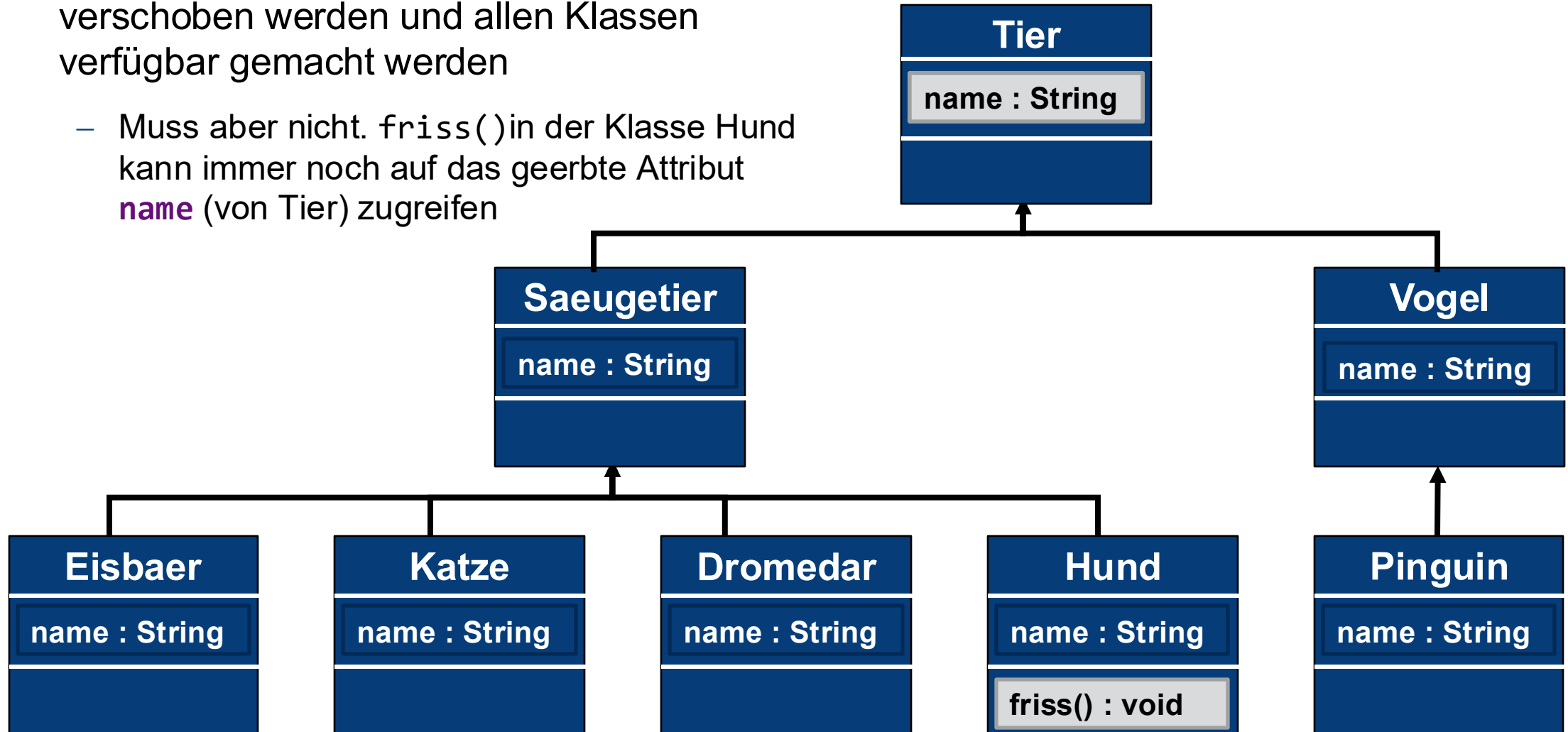
Noch eine Ebene

- Die Methode `friss()`, die auf das Attribut **name** angewiesen ist, kann genauso verschoben werden und allen Klassen verfügbar gemacht werden



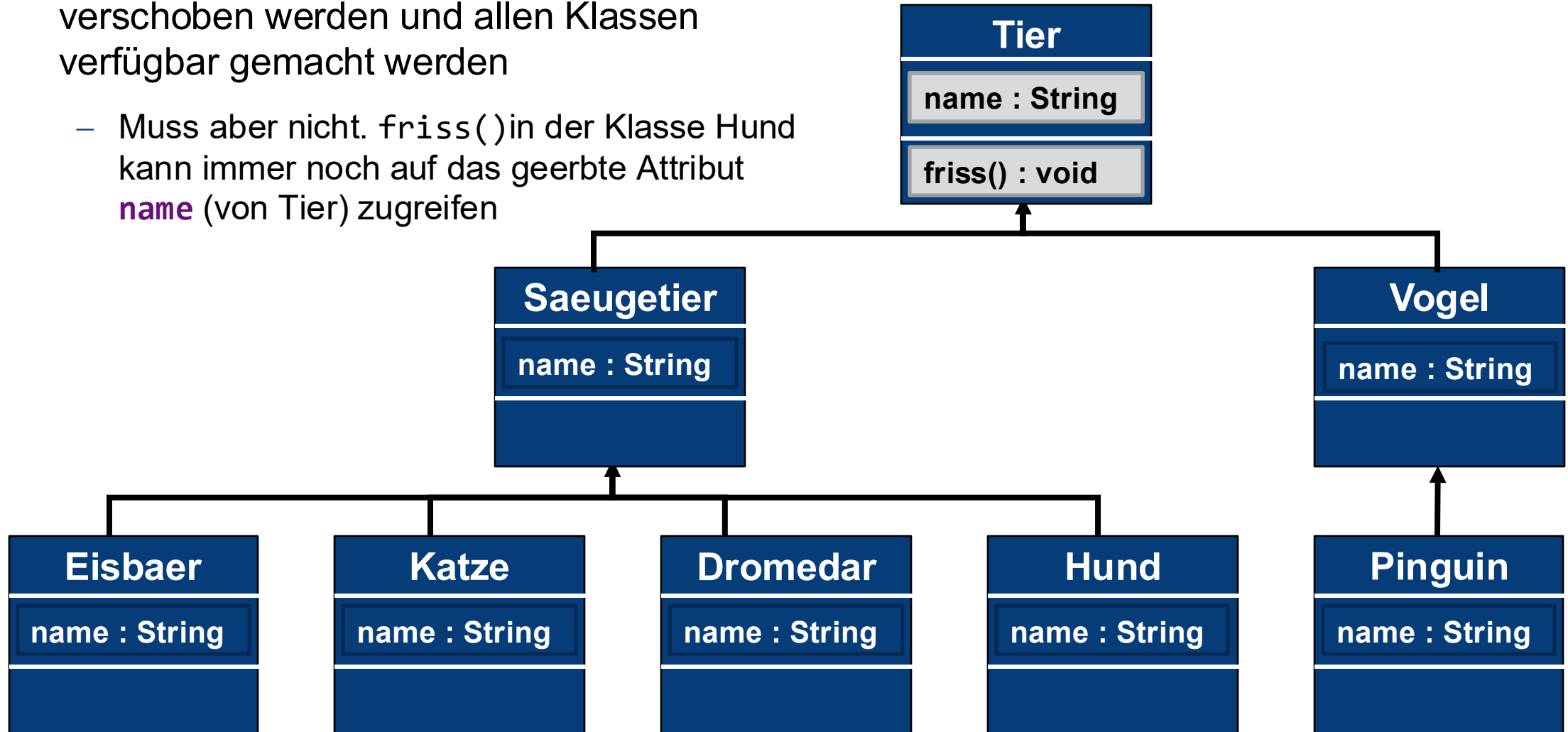
Noch eine Ebene

- Die Methode `friss()`, die auf das Attribut **name** angewiesen ist, kann genauso verschoben werden und allen Klassen verfügbar gemacht werden
 - Muss aber nicht. `friss()` in der Klasse `Hund` kann immer noch auf das geerbte Attribut **name** (von `Tier`) zugreifen



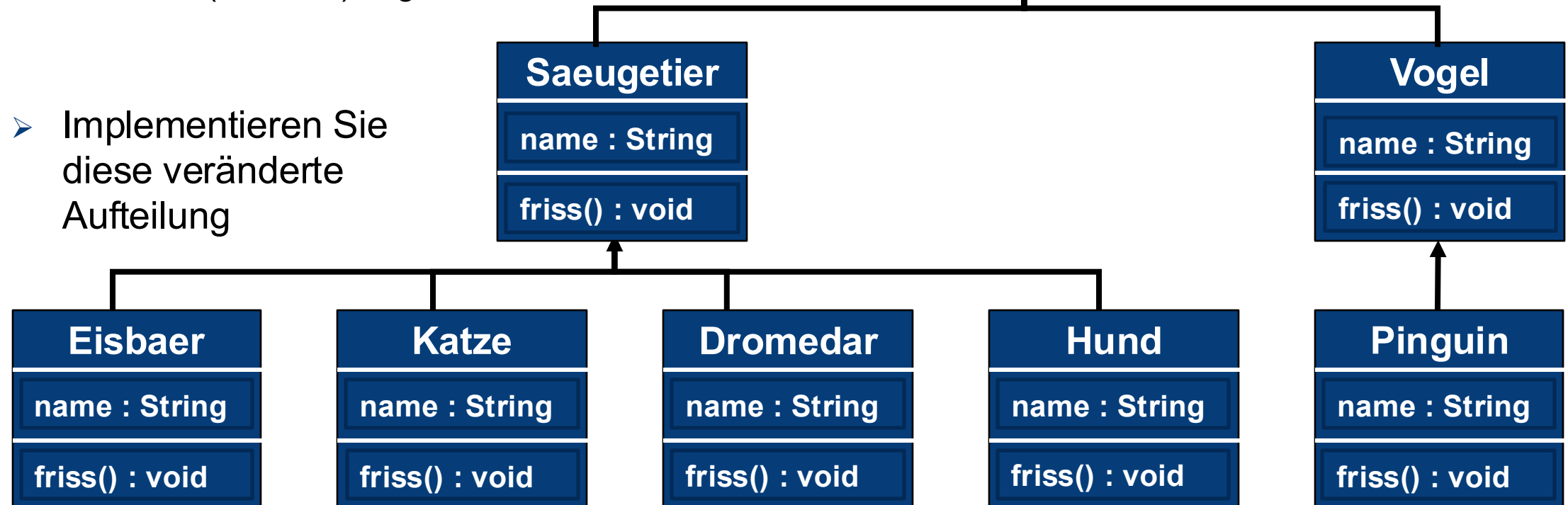
Noch eine Ebene

- Die Methode `friss()`, die auf das Attribut **name** angewiesen ist, kann genauso verschoben werden und allen Klassen verfügbar gemacht werden
 - Muss aber nicht. `friss()` in der Klasse `Hund` kann immer noch auf das geerbte Attribut **name** (von `Tier`) zugreifen



Noch eine Ebene

- Die Methode `friss()`, die auf das Attribut **name** angewiesen ist, kann genauso verschoben werden und allen Klassen verfügbar gemacht werden
 - Muss aber nicht. `friss()` in der Klasse `Hund` kann immer noch auf das geerbte Attribut **name** (von `Tier`) zugreifen



Noch eine Ebene

Musterlösung:

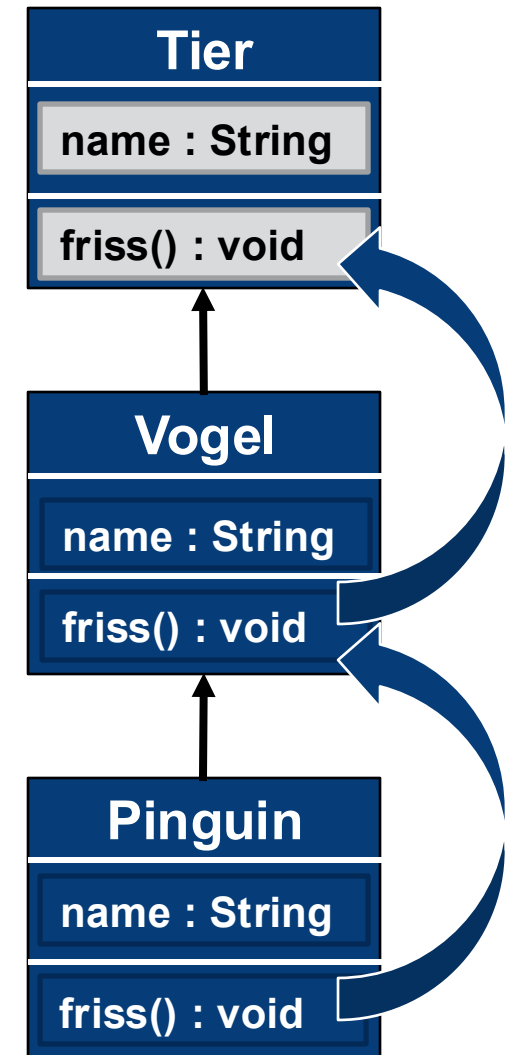
```
1 public class Tier {  
2  
3     String name;  
4  
5     public void friss() {  
6         System.out.println(name + ": *mampf*");  
7     }  
8 }
```

@Override

ÜBERSCHREIBEN (POLYMORPHISMUS)

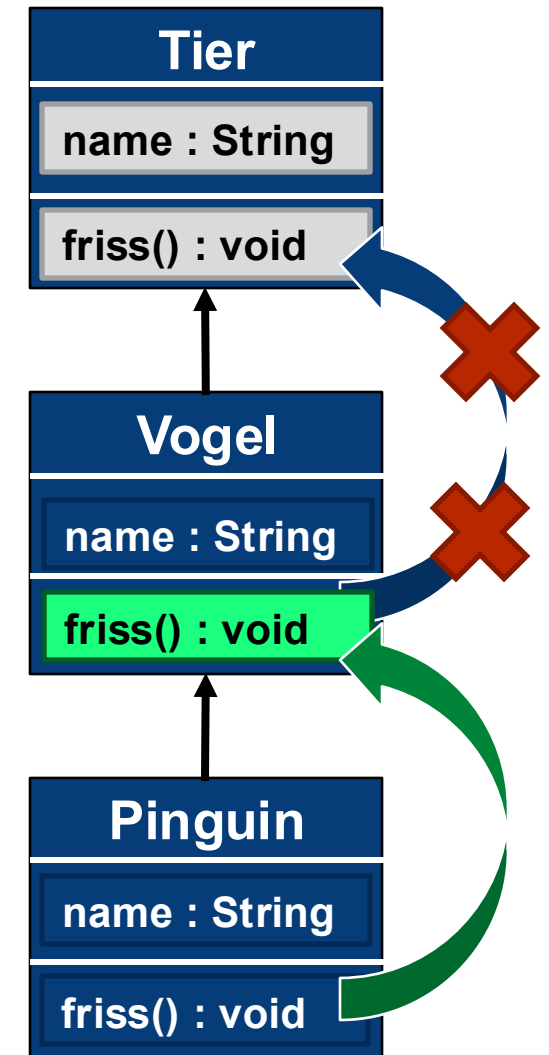
Polymorphismus: Methoden überschreiben

- Nicht alle Tiere fressen auf die gleiche Art und Weise
- Deswegen passen wir die Methode `friss()` bei manchen Tierarten an
- Zunächst sollen alle Vögel nicht **mampf**, sondern **pick** sagen
- Dafür wird die Methode `friss()` einfach in der Klasse `Vogel` neu definiert



Polymorphismus: Methoden überschreiben

- Nicht alle Tiere fressen auf die gleiche Art und Weise
 - Deswegen passen wir die Methode `friss()` bei manchen Tierarten an
 - Zunächst sollen alle Vögel nicht **mampf**, sondern **pick** sagen
 - Dafür wird die Methode `friss()` einfach in der Klasse `Vogel` neu definiert
 - Die Kindklassen von `Vogel` benutzen dann die Implementierung aus `Vogel`, nicht die aus `Tier`
- Überschreiben Sie die Methode `friss()` in der Klasse `Vogel` und Testen Sie was beim Aufruf von `friss()` auf einer Instanz von `Pinguin` passiert



Polymorphismus: Methoden überschreiben

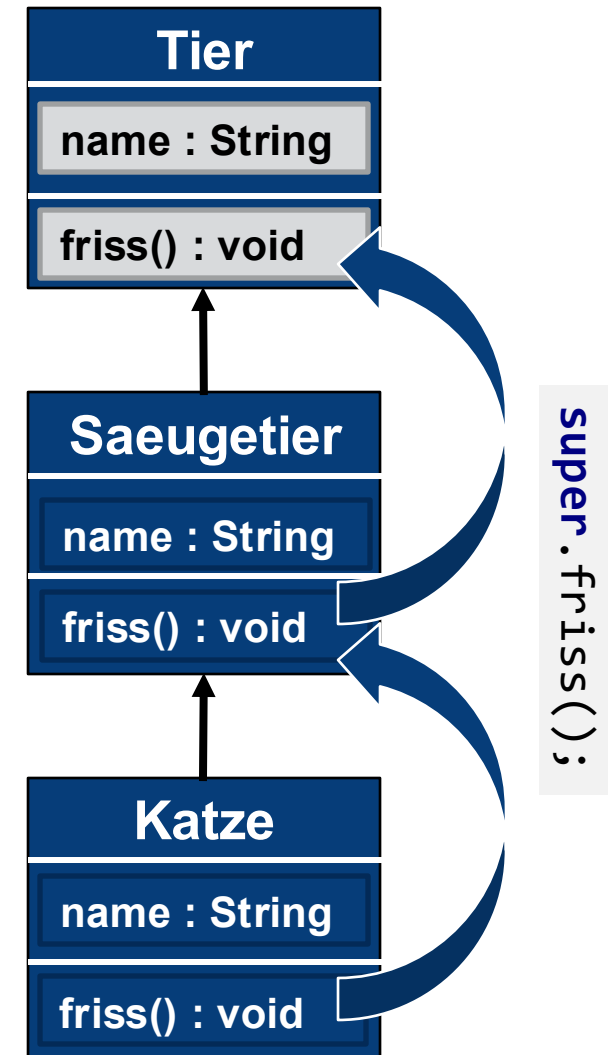
Musterlösung:

@Override
ist optional

```
1 public class Vogel extends Tier {  
2     @Override  
3     public void friss() {  
4         System.out.println(name + ": *pick*");  
5     }  
6 }
```

Die schnurrende Katze

- Als nächstes passen wir die Klasse Katze an
 - Katzen sollen **zusätzlich** zu ihrem normalen (geerbten) Verhalten beim Fressen ein *schnurr* von sich geben
 - Dafür wird innerhalb der überschreibenden Methode friss() (in Katze) die geerbte Version von friss() (aus Tier) durch **super**.friss(); aufgerufen
- Überschreiben Sie die Methode friss() in Katze und nutzen Sie dabei **super**



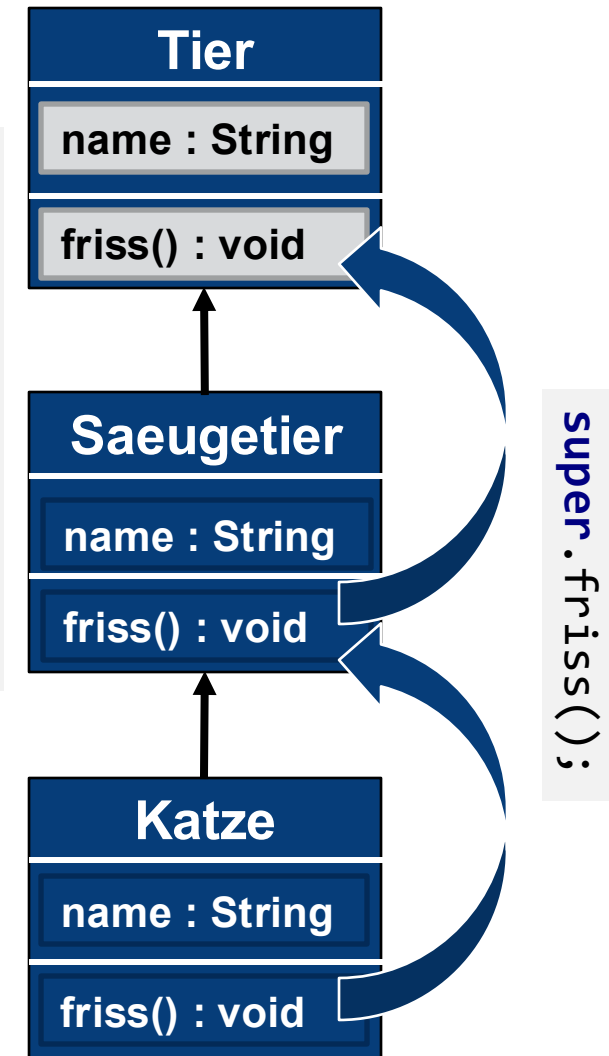
Die schnurrende Katze

Musterlösung:

```
1 public class Katze extends Saeugetier {
2     @Override
3     public void friss() {
4         super.friss();
5         System.out.println(name + ": *schnurr*");
6     }
7 }
```

@Override
ist optional

- `super.friss()` ruft die nächsthöhere Implementierung von `friss()` auf
- `Saeugetier` überschreibt `friss()` nicht, deswegen wird die Version von `Tier` verwendet

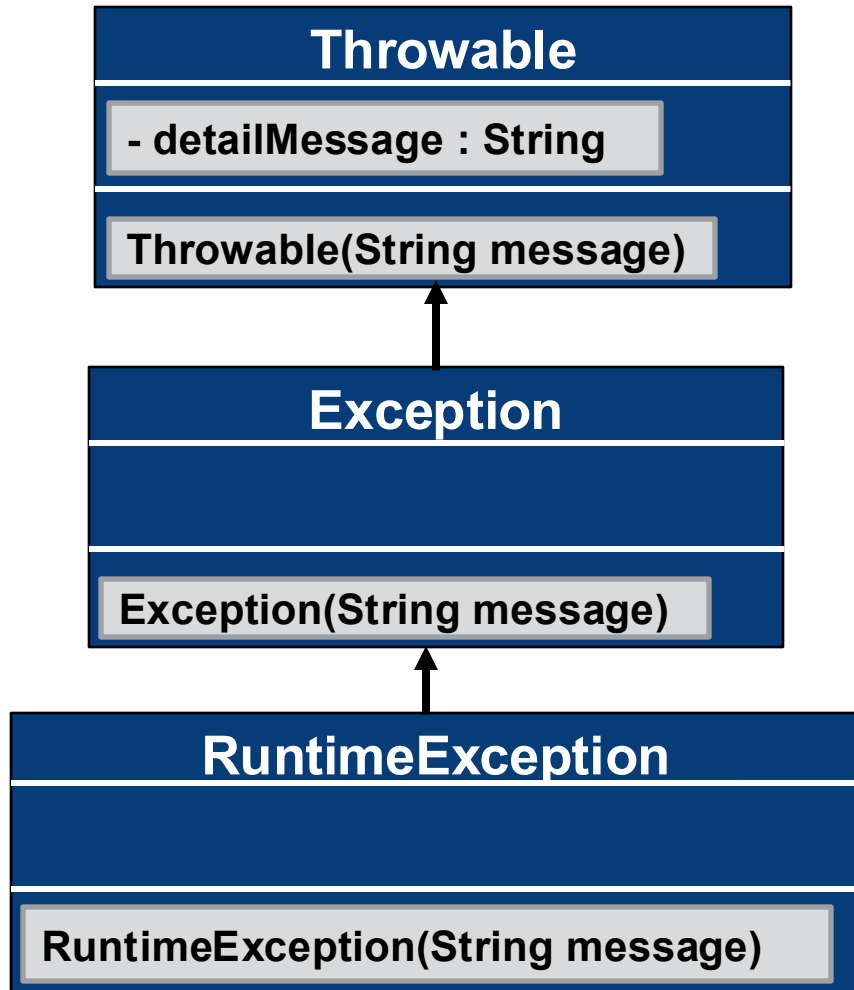


super bei Konstruktoren

- Wir haben jetzt gesehen, wie man Methoden überschreibt und mit **super** auf die nächsthöhere Implementierung einer Methode zugreift
- Das gleiche Vorgehen ist auch beim Konstruktor mit dem Aufruf von **super()** möglich
- Beispiel aus der Java API: Exceptions (s. nächste Folie)

super bei Konstruktoren

Beispiel aus der Java API: Exceptions



```
private String detailMessage;
public Throwable(String message) {
    fillInStackTrace();
    detailMessage = message;
}
```

```
public Exception(String message) {
    super(message);
}
```

```
public RuntimeException(String message) {
    super(message);
}
```

- Diese Art der Verwendung hat verschiedene Vorteile
 - Aufwendige Initialisierungen (z.B. von Attributen) müssen nur in einer Mutterklasse gemacht werden (Vorteil: Wiederverwenden von Code)
 - Attribute müssen nicht als `protected` gekennzeichnet werden, sondern können auch `private` sein (siehe `detailMessage`) (Vorteil: Noch mehr Data Hiding)
- Fallstricke
 - `super` muss in Konstruktoren immer als erste Anweisung stehen
 - `super` kann nur einmal im Konstruktor aufgerufen werden
 - Der Aufruf enthält keinen Punkt, sondern nach `super` kommt direkt die Klammer mit den Parametern

VORTEILE DER VERERBUNG

Ein Zoo

- Die Vererbung erlaubt es uns, konkrete Tierarten (Unterklassen) anstelle von direkten Instanzen von Tier (Oberklasse) zu nutzen
- Dies ist u. a. bei Arrays und Collections nützlich
- Ein Tier-Array kann Instanzen von Tier, aber auch Instanzen von Klassen, die von Tier erben enthalten

**Mehr zu
Collections
später**

Ein Zoo

```
1 Eisbaer eisbaer1 = new Eisbaer();
2 eisbaer1.name = "Knut";
3 Katze katze1 = new Katze();
4 katze1.name = "Miez";
5 Dromedar dromedar1 = new Dromedar();
6 dromedar1.name = "Dromo";
7 Hund hund1 = new Hund("Strolch");
8 Pinguin pinguin1 = new Pinguin();
9 pinguin1.name = "Pingo";
10
11 Tier[] zoo = new Tier[] {
12     eisbaer1,
13     katze1,
14     dromedar1,
15     hund1,
16     pinguin1
17 };
18
19 for (int i = 0; i < zoo.length; i++) {
20     zoo[i].friss();
21 }
```

Ausgabe:

```
Knut: *mampf*
Miez: *mampf*
Miez: *schnurr*
Dromo: *mampf*
Strolch: *mampf*
Pingo: *pick*
```

LÖSUNG SCHRIFTLICHES AUFGABENBLATT 2

Lösung Aufgabenblatt 2

- **Nicht Teil des Skripts – wird nur in der Übung besprochen**

Ausblick

PROGRAMMIERAUFGABE 3

Programmieraufgabe 3

- Problem: Implementieren Sie einen Ableitungsrechner für Funktionen
- Wichtig:
 - Kompositum Entwurfsmuster, um alle Funktionen gleich zu verwenden (Interface)
 - Unveränderbarkeit durch *immutable* Objects. Eine Änderung eines Objektes erzeugt eine Kopie, die die Änderung beinhaltet.
 - Basisfunktionen richtig implementieren! Testen Sie selbstständig lokal Ihre Implementierung.
- Abgabe im neuen Jahr bis zum 16.01. um 11 Uhr

**Frohe Weihnachten und einen guten
Rutsch ins neue Jahr!**

